CORDIC Algorithm

- **RTL code**

```
module CORDIC (clk,x_start,y_start,angle,sin,cos);

// Full circle (360°) = (2^32) units

// fixed angle = angle_degree * (2*32)/360


parameter WIDTH = 16;

//
localparam ITER = WIDTH;

input clk;
input signed [WIDTH-1:0] x_start,y_start;
input signed [31:0] angle;
output signed [WIDTH-1:0] sin,cos;   // 1_bit larger than x0,yo due to gain (1.647)

 wire signed [31:0] atan_table [0:30];

 // THE upper tow bits of the angle is used to know which quadrant
 // 2'b00 --> 0 -->pi/2 range
 // 2'b01 --> pi/2-->pi range
 // 2'b10 --> pi-->3pi/2 range (-pi/2--> -pi)
 // 2'b11 --> 3pi/2-->2pi range (0--> -pi/2)


  assign atan_table[00] = 'b00100000000000000000000000000000; // 45.000 degrees -> atan(2^0)
  assign atan_table[01] = 'b00010010111001000000010100011101; // 26.565 degrees  -> atan(2^ 1)
  assign atan_table[02] = 'b00001001111110110011100001011011; // 14.036 degrees -> atan(2^ 2)
  assign atan_table[03] = 'b00000101000100010001000111010100; // atan(2^-3)
  assign atan_table[04] = 'b00000010100010110000110101000011;
  assign atan_table[05] = 'b00000001010001011101011111100001;
  assign atan_table[06] = 'b00000000101000101111011000011110;
  assign atan_table[07] = 'b00000000010100010111110001010101;
  assign atan_table[08] = 'b00000000001010001011111001010011;
  assign atan_table[09] = 'b00000000000101000101111100101110;
  assign atan_table[10] = 'b00000000000010100010111110011000;
  assign atan_table[11] = 'b00000000000001010001011111001100;
  assign atan_table[12] = 'b00000000000000101000101111100110;
  assign atan_table[13] = 'b00000000000000010100010111110011;
  assign atan_table[14] = 'b00000000000000001010001011111001;
  assign atan_table[15] = 'b00000000000000000101000101111100;
  assign atan_table[16] = 'b00000000000000000010100010111110;
  assign atan_table[17] = 'b00000000000000000001010001011111;
  assign atan_table[18] = 'b00000000000000000000101000101111;
  assign atan_table[19] = 'b00000000000000000000010100010111;
  assign atan_table[20] = 'b00000000000000000000001010001011;
  assign atan_table[21] = 'b00000000000000000000000101000101;
  assign atan_table[22] = 'b00000000000000000000000010100010;
  assign atan_table[23] = 'b00000000000000000000000001010001;
  assign atan_table[24] = 'b00000000000000000000000000101000;
  assign atan_table[25] = 'b00000000000000000000000000010100;
  assign atan_table[26] = 'b00000000000000000000000000001010;
  assign atan_table[27] = 'b00000000000000000000000000000101;
  assign atan_table[28] = 'b00000000000000000000000000000010;
  assign atan_table[29] = 'b00000000000000000000000000000001;
  assign atan_table[30] = 'b00000000000000000000000000000000;


    //
    reg signed [WIDTH-1:0] x_reg [0:ITER-1];  // for pipelining
    reg signed [WIDTH-1:0] y_reg [0:ITER-1];  // for pipelining
    reg signed [31:0] z_reg [0:ITER-1];    // for pipelining
    // due to piplining we have result every clock cycle


    wire  [1:0] quadrant;

    assign quadrant = angle[31:30];


    always @(posedge clk) begin
       case (quadrant)
        2'b00,2'b11 : begin
            x_reg[0] <= x_start;
            y_reg[0] <= y_start;
            z_reg[0] <= angle;
        end
        2'b01: begin
            // rotate 90 degree
            x_reg[0] <= -y_start;
            y_reg[0] <= x_start;
            z_reg[0] <= {2'b01,angle[29:0]};
        end
        2'b10: begin
            x_reg[0] <= y_start;
            y_reg[0] <= -x_start;
            z_reg[0] <= {2'b10,angle[29:0]};
        end
       endcase
    end

    genvar i;
    generate
        for (i = 0; i<(ITER-1); i = i + 1) begin
            wire  sign_of_z;
            assign sign_of_z = z_reg[i][31];
            always @(posedge clk) begin
                x_reg[i+1] <= (sign_of_z)? x_reg[i] + (y_reg[i] >>> i)  : x_reg[i] - (y_reg[i] >>> i);
                y_reg[i+1] <= (sign_of_z)? y_reg[i] - (x_reg[i] >>> i)  : y_reg[i] + (x_reg[i] >>> i);
                z_reg[i+1] <= (sign_of_z)? z_reg[i] + atan_table[i]  : z_reg[i]  atan_table[i];
            end
        end

    endgenerate


//output
      assign cos = x_reg[ITER-1];  //xn
      assign sin = y_reg[ITER-1];  //yn


endmodule //CORDIC
```

```
module tb (

);
parameter WIDTH = 16;
reg clk;
reg [WIDTH-1:0] x_start,y_start;
reg [31:0] angle;
wire [WIDTH-1:0] sin,cos;
//
localparam An = 19436;    //


//
CORDIC  DUT(.*);
//
initial begin
    clk = 1'b0;
    forever   #1 clk = ~clk;
end
//
initial begin
   x_start = An; y_start = 0;
   // 45
   angle =  'b00100000000000000000000000000000;
   repeat(18) begin
     @(negedge clk);
   end
   // 60
   angle =  'b00101010101010101010101010101010;
   repeat(18) begin
     @(negedge clk);
   end
   // 90
   angle =  'b01000000000000000000000000000000;
   repeat(18) begin
     @(negedge clk);
   end
   // 75
   angle =  'b00110101010101010101010101010101;
   repeat(18) begin
     @(negedge clk);
   end
   // 0
   angle =  32'b0;
   repeat(18) begin
     @(negedge clk);
   end

    $stop;

end

endmodule //tb
```

DO file

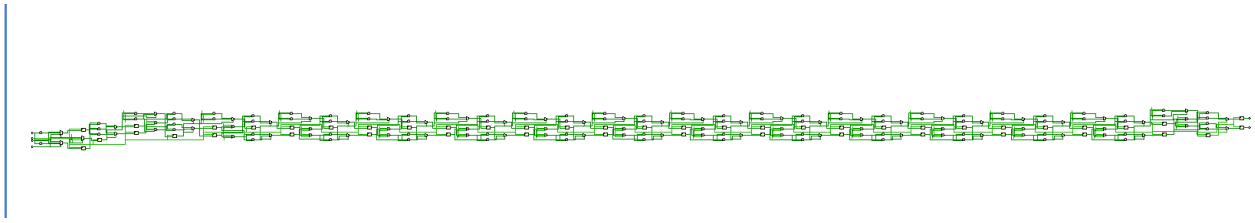
run_cordic.do

```
vlib work
vlog CORDIC.v tb.v
vsim -voptargs=+acc work.tb
add wave *
run -all
#quit -sim
```

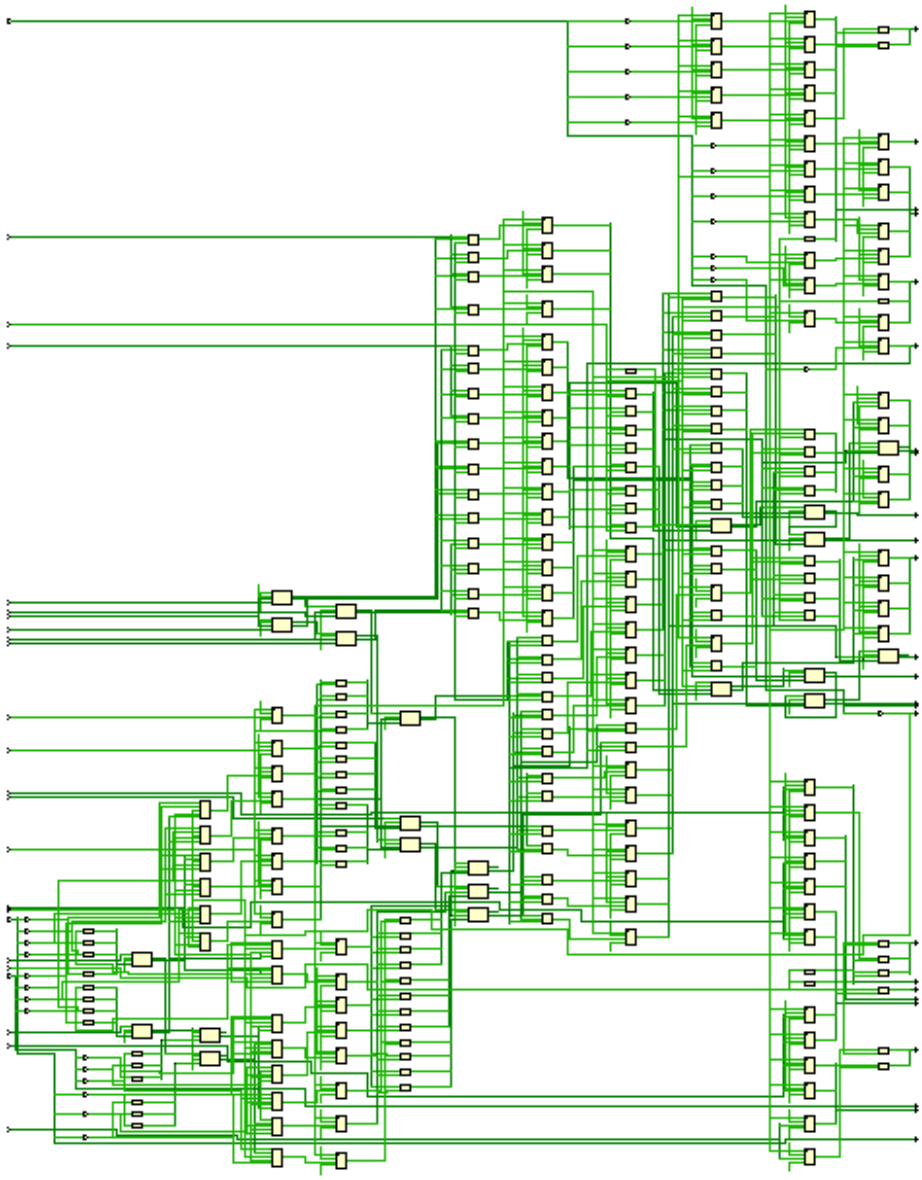
Constrain file

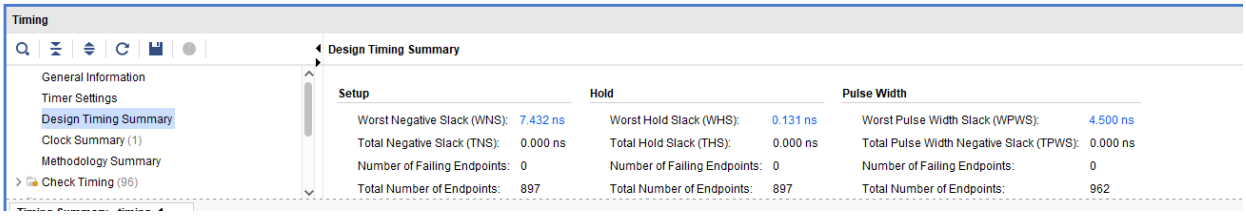
```
## Clock signal
set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
```

Vivado flow



Synthesis




Timing
General Information
Timer Settings
Design Timing Summary
Clock Summary (1)
Methodology Summary
Check Timing (96)
Design Timing Summary
Setup
Worst Negative Slack (WNS): 7.432 ns
Total Negative Slack (TNS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 897
Hold
Worst Hold Slack (WHS): 0.131 ns
Total Hold Slack (THS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 897
Pulse Width
Worst Pulse Width Slack (WPWS): 4.500 ns
Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 962

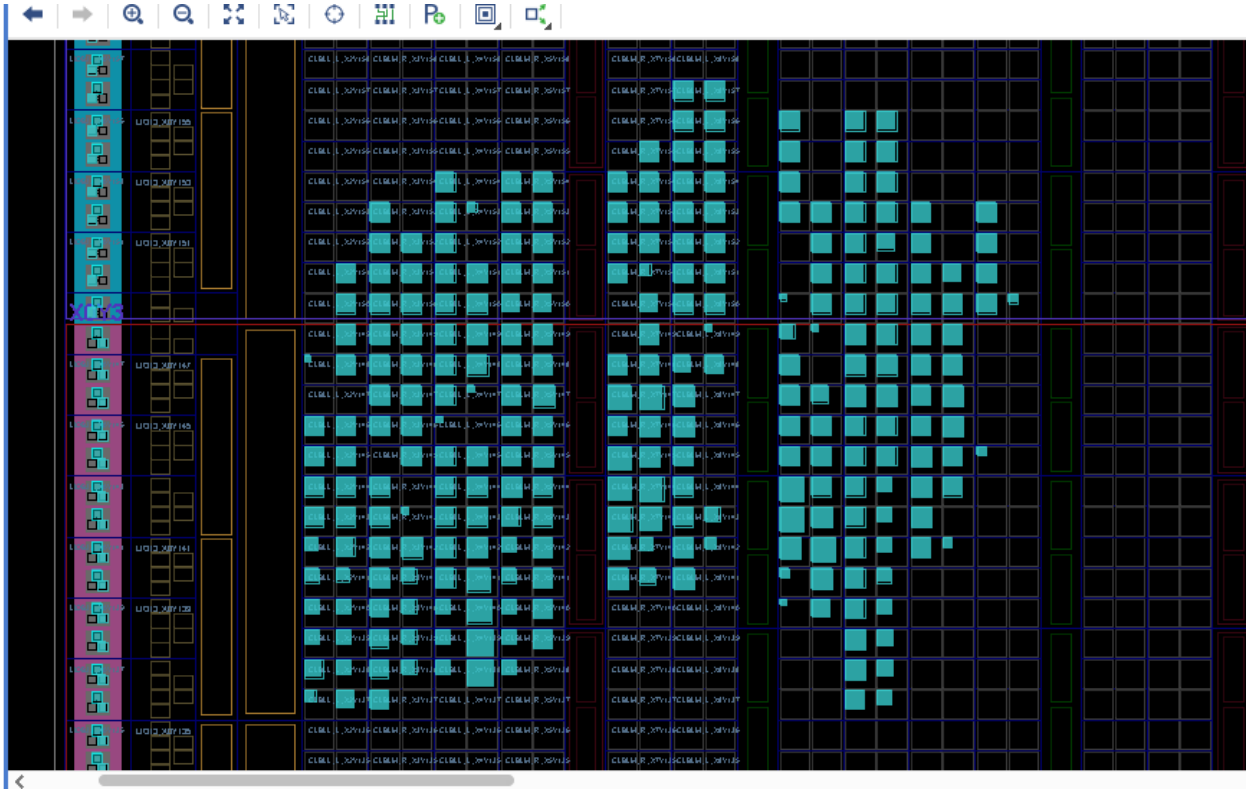
Timing | Utilization

Hierarchy
Summary
Slice Logic
Slice LUTs (1%)
LUT as Logic (1%)
Slice Registers (1%)

Hierarchy

Name	Slice LUTs (134600)	Slice Registers (269200)	Bonded IOB (500)	BUFGCTRL (32)
CORDIC	997	961	97	1

Implementation



Tcl Console | Messages | Log | Reports | Design Runs | DRC | Methodology | Power | Timing

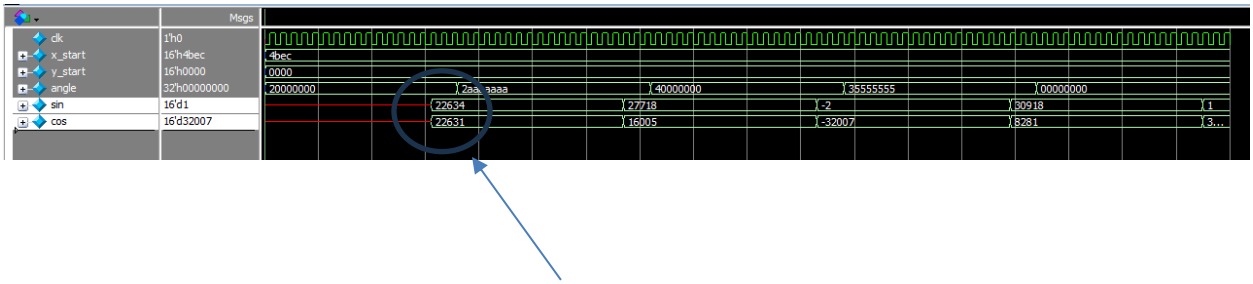
General Information
Timer Settings
Design Timing Summary
Clock Summary (1)
Methodology Summary (96)
Check Timing (96)
Intra-Clock Paths

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	6.715 ns	Worst Hold Slack (WHS):	0.057 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	897	Total Number of Endpoints:	897	Total Number of Endpoints:	962

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Bonded IOB (500)	BUFGCTRL (32)
CORDIC	997	961	269	997	97	1

Simulation



To map the fixed-point outputs into the standard range [−1,+1] each raw output must be divided by the amplitude factor A=32010A